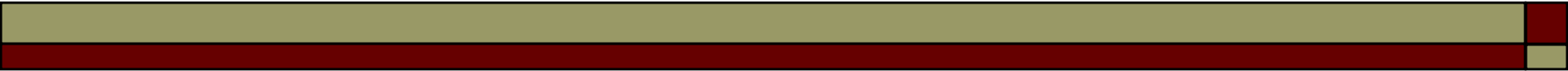


Software Construction and Development SE-312

LECTURE # 07

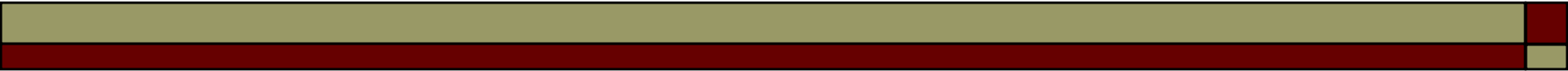
Code Salvaging and Configuration Management

Course Instructor: Engr. Ruqqiya Asad



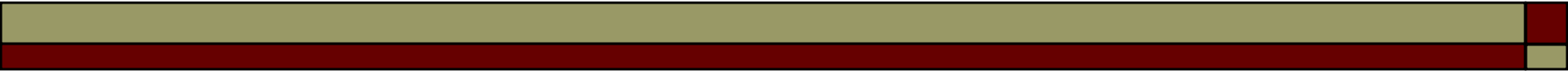
Code Salvaging

Code Salvaging is the process of:
Reusing useful parts of existing code from old or legacy software in new software projects instead of writing everything from scratch.
It is a **software maintenance technique** to save time, cost, and effort.



Objectives / Purpose

- **Save Time:** Avoid rewriting modules that already work.
- **Reduce Cost:** Less development and testing required.
- **Reuse Tested Code:** Modules that were already tested can reduce errors.
- **Accelerate Development:** Faster delivery of new system



Types of Code Salvaging

Complete Module Salvage: Entire module reused without change.

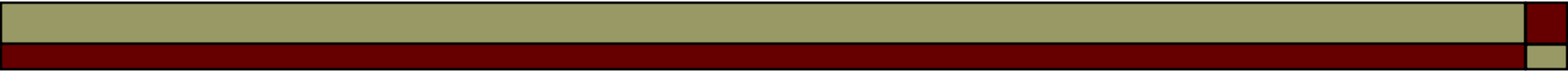
Example: Tax calculation module from old billing system.

Partial Salvage: Some functions or classes reused.

Example: Authentication logic reused but with new UI.

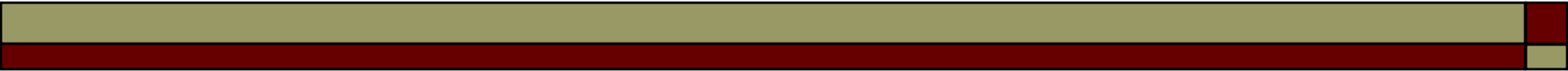
Code Refactoring: Old code is modified for new system but logic remains.

Example: Legacy payroll logic adapted to new HR system.



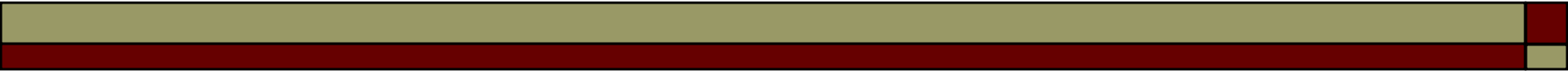
When is Code Salvaging Useful?

- Legacy system modernization
- System migration (e.g., desktop → web)
- Upgrading software without losing working features
- Large projects where rewriting is expensive



Challenges in Code Salvaging

- **Poor Documentation:** Old code may lack comments.
- **Compatibility Issues:** Old code may not work in new language or framework.
- **Tightly Coupled Code:** Hard to extract specific modules.
- **Hidden Bugs:** Old code may have undetected issues.



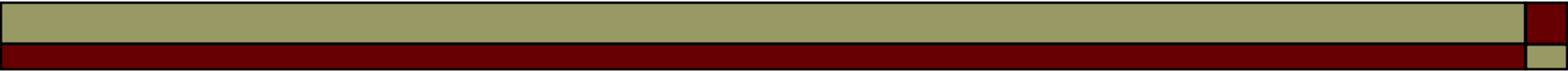
Example Scenario

Situation: A bank has a 15-year-old desktop software.

Goal: Develop a mobile app.

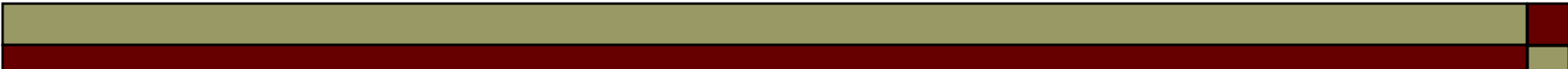
- Salvaged code: Transaction validation & security module
- Rewritten: User interface, reports, and mobile-specific features

This avoids rewriting complex logic like transaction security from scratch.



Steps in Code Salvaging

- **Identify modules to salvage** – analyze old system to find reusable parts.
- **Check compatibility** – ensure module fits in new system architecture.
- **Refactor if necessary** – modify code to meet new requirements.
- **Integrate into new system** – test salvaged modules with new code.
- **Document changes** – record what is reused and what is modified.



Configuration Management (CM)

Configuration Management is:

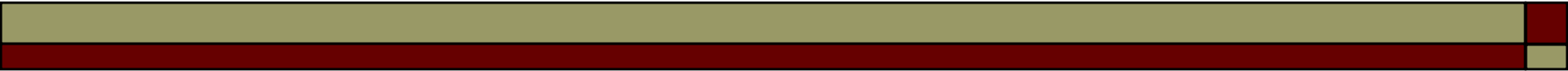
The process of systematically managing software versions, changes, and documentation to ensure controlled development and maintenance.

It ensures **organized evolution of software**, especially in multi-developer environments.

Configuration management



- ✧ Software systems are constantly changing during development and use.
- ✧ Configuration management (CM) is concerned with the policies, processes and tools for managing changing software systems.
- ✧ You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version.
- ✧ CM is essential for team projects to control changes made by different developers



Objectives

- 1.Track changes** – every modification is recorded.
- 2.Maintain versions** – rollback if a change causes errors.
- 3.Coordinate team development** – multiple developers can work without conflict.
- 4.Ensure stability** – prevent accidental overwriting of working code.
- 5.Improve documentation** – all changes are traceable.

Key Activities of Configuration Management

2.3.1 Version Control

- Track history of changes in code, documents, and configuration files.
- Rollback to previous versions if needed.

Tools: Git, SVN

Example:

Version 1.0 → Initial system

Version 1.1 → Bug fixes

Version 2.0 → Major feature update

Version management



- ✧ Version management (VM) is the process of keeping track of different versions of software components or configuration items and the systems in which these components are used.
- ✧ It also involves ensuring that changes made by different developers to these versions do not interfere with each other.
- ✧ Therefore version management can be thought of as the process of managing codelines and baselines.

Codelines and baselines



- ✧ A codeline is a sequence of versions of source code with later versions in the sequence derived from earlier versions.
- ✧ Codelines normally apply to components of systems so that there are different versions of each component.
- ✧ A baseline is a definition of a specific system.
- ✧ The baseline therefore specifies the component versions that are included in the system plus a specification of the libraries used, configuration files, etc.

Baselines



- ✧ Baselines may be specified using a configuration language, which allows you to define what components are included in a version of a particular system.
- ✧ Baselines are important because you often have to recreate a specific version of a complete system.
 - For example, a product line may be instantiated so that there are individual system versions for different customers. You may have to recreate the version delivered to a specific customer if, for example, that customer reports bugs in their system that have to be repaired.

Codelines and baselines



Codeline (A)



Codeline (B)



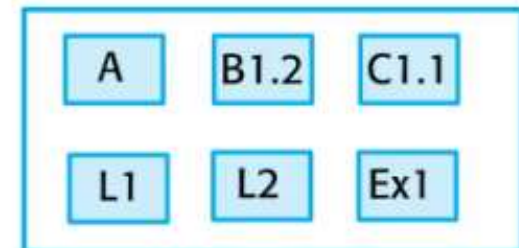
Codeline (C)



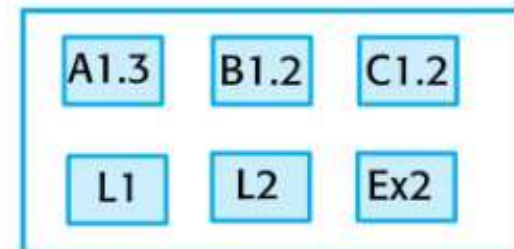
Libraries and external components



Baseline - V1



Baseline - V2



Mainline

Public repository and private workspaces



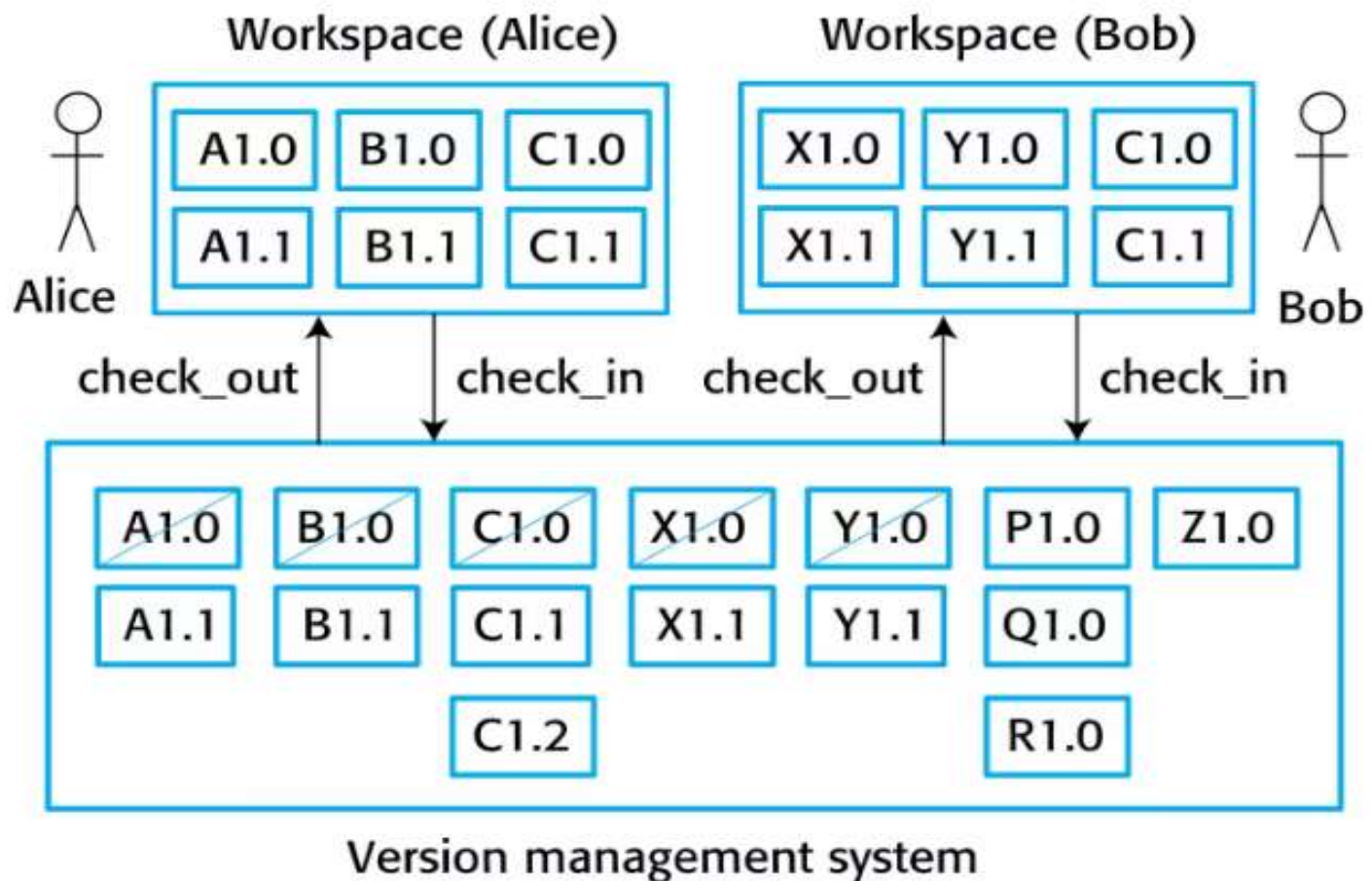
- ✧ To support independent development without interference, version control systems use the concept of a project repository and a private workspace.
- ✧ The project repository maintains the 'master' version of all components. It is used to create baselines for system building.
- ✧ When modifying components, developers copy (check-out) these from the repository into their workspace and work on these copies.
- ✧ When they have finished their changes, the changed components are returned (checked-in) to the repository.

Centralized version control



- ✧ Developers check out components or directories of components from the project repository into their private workspace and work on these copies in their private workspace.
- ✧ When their changes are complete, they check-in the components back to the repository.
- ✧ If several people are working on a component at the same time, each check it out from the repository. If a component has been checked out, the VC system warns other users wanting to check out that component that it has been checked out by someone else.

Repository Check-in/Check-out

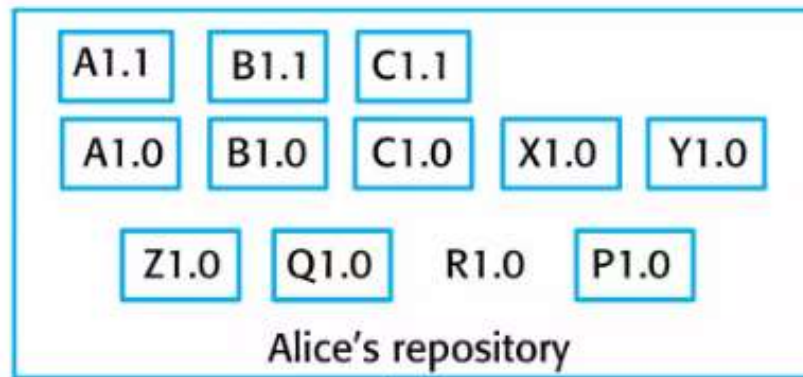


Distributed version control

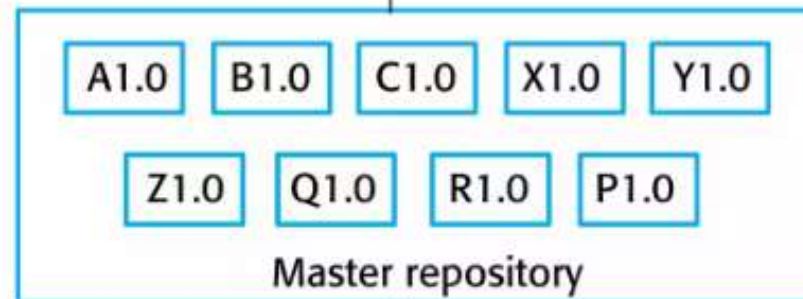


- ✧ A 'master' repository is created on a server that maintains the code produced by the development team.
- ✧ Instead of checking out the files that they need, a developer creates a clone of the project repository that is downloaded and installed on their computer.
- ✧ Developers work on the files required and maintain the new versions on their private repository on their own computer.
- ✧ When changes are done, they 'commit' these changes and update their private server repository. They may then 'push' these changes to the project repository.

Alice

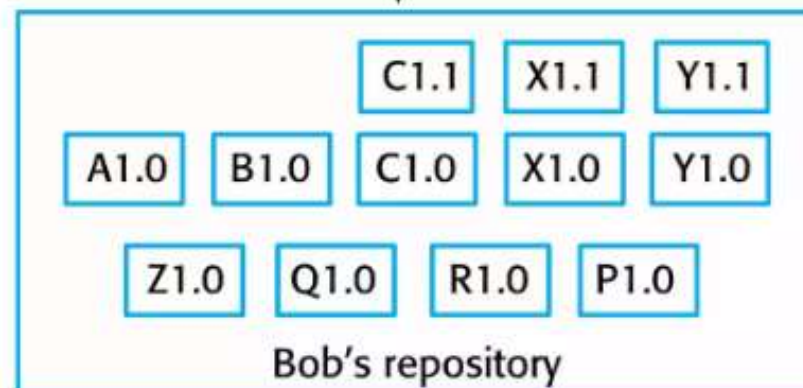


clone



clone

Bob



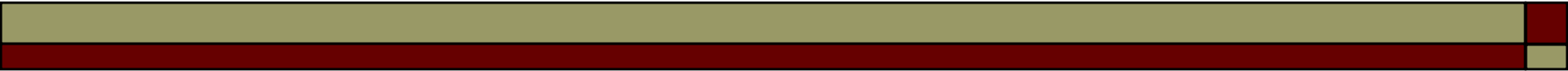
Repository cloning



Benefits of distributed version control



- ✧ It provides a backup mechanism for the repository.
 - If the repository is corrupted, work can continue and the project repository can be restored from local copies.
- ✧ It allows for off-line working so that developers can commit changes if they do not have a network connection.
- ✧ Project support is the default way of working.
 - Developers can compile and test the entire system on their local machines and test the changes that they have made.



Change Control

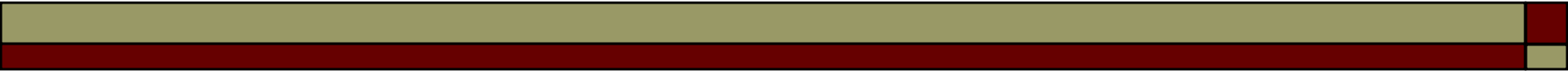
- All changes must be **planned, approved, and documented.**

Example:

A developer cannot directly change production code.

Steps:

1. Submit change request
2. Manager approves
3. Merge into main branch

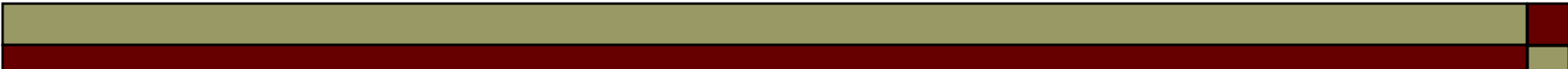


Build Management

- Compiling and packaging software for testing or deployment.

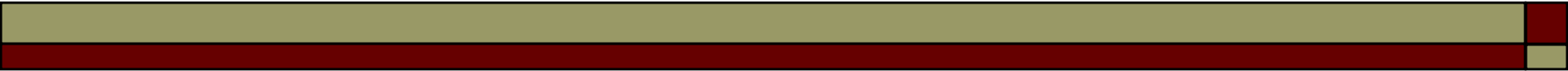
Example:

- Daily build of web application
- Mobile APK generation
- Executable for Windows desktop



Build system functionality

- ✧ Build script generation
- ✧ Version management system integration
- ✧ Minimal re-compilation
- ✧ Executable system creation
- ✧ Test automation
- ✧ Reporting
- ✧ Documentation generation



Release Management

- Deploying software versions to end-users.

Example:

- Release 1.0 → basic functionality
- Release 1.1 → bug fixes
- Release 2.0 → new features

Release management



- ✧ A system release is a version of a software system that is distributed to customers.
- ✧ For mass market software, it is usually possible to identify two types of release: major releases which deliver significant new functionality, and minor releases, which repair bugs and fix customer problems that have been reported.
- ✧ For custom software or software product lines, releases of the system may have to be produced for each customer and individual customers may be running several different releases of the system at the same time.

Release components



- ✧ As well as the the executable code of the system, a release may also include:
 - configuration files defining how the release should be configured for particular installations;
 - data files, such as files of error messages, that are needed for successful system operation;
 - an installation program that is used to help install the system on target hardware;
 - electronic and paper documentation describing the system;
 - packaging and associated publicity that have been designed for that release.

Release creation



- ✧ The executable code of the programs and all associated data files must be identified in the version control system.
- ✧ Configuration descriptions may have to be written for different hardware and operating systems.
- ✧ Update instructions may have to be written for customers who need to configure their own systems.
- ✧ Scripts for the installation program may have to be written.
- ✧ Web pages have to be created describing the release, with links to system documentation.
- ✧ When all information is available, an executable master image of the software must be prepared and handed over for distribution to customers or sales outlets.

Release tracking



- ✧ In the event of a problem, it may be necessary to reproduce exactly the software that has been delivered to a particular customer.
- ✧ When a system release is produced, it must be documented to ensure that it can be re-created exactly in the future.
- ✧ This is particularly important for customized, long-lifetime embedded systems, such as those that control complex machines.
 - Customers may use a single release of these systems for many years and may require specific changes to a particular software system long after its original release date.

Release reproduction



- ✧ To document a release, you have to record the specific versions of the source code components that were used to create the executable code.
- ✧ You must keep copies of the source code files, corresponding executables and all data and configuration files.
- ✧ You should also record the versions of the operating system, libraries, compilers and other tools used to build the software.

Release planning



- ✧ As well as the technical work involved in creating a release distribution, advertising and publicity material have to be prepared and marketing strategies put in place to convince customers to buy the new release of the system.
- ✧ Release timing
 - If releases are too frequent or require hardware upgrades, customers may not move to the new release, especially if they have to pay for it.
 - If system releases are too infrequent, market share may be lost as customers move to alternative systems.